Arquitetura serverless na AWS com infraestrutura como código e entrega contínua: o caso QuizOps

Serverless architecture on AWS with infrastructure as code and continuous delivery: the QuizOps case

Luiz Gustavo Gonçalves Brito*

2026

Resumo

Este artigo descreve o projeto e a implementação do QuizOps, uma aplicação web de quiz com ranking e mural de fotos, construída sobre serviços gerenciados da Amazon Web Services (AWS). O objetivo foi obter uma aplicação completa, com autenticação, armazenamento de arquivos e entrega automatizada, sem servidores para administrar e com custo proporcional ao uso. A infraestrutura foi descrita em Terraform, organizada em três camadas (front-end, back-end e dados), e o código é publicado por pipelines do GitHub Actions. O front-end em React é servido pelo Amazon CloudFront a partir de um bucket privado do Amazon S3; a autenticação usa o Amazon Cognito com o protocolo SRP; um API Gateway HTTP valida o token JWT antes de invocar uma única função AWS Lambda, que persiste os dados no Amazon DynamoDB e emite URLs pré-assinadas para o envio direto de fotos ao S3. Os resultados mostram que a solução atende aos requisitos funcionais, pode ser recriada em outra conta com poucos comandos e removida por completo com um único comando. São discutidas as decisões de segurança e custo, as limitações encontradas e a correspondência dos serviços com o Google Cloud, o que indica a portabilidade da arquitetura em um contexto multicloud.

Palavras-chave: computação em nuvem; serverless; infraestrutura como código; Terraform; AWS.

*Aluno do curso de Cloud Computing da FIAP, RM 562192. E-mail: luizg1717@gmail.com.

Abstract

This paper describes the design and implementation of QuizOps, a quiz web application with a leaderboard and a photo wall, built on Amazon Web Services (AWS) managed services. The goal was a complete application, with authentication, file storage and automated delivery, without servers to manage and with costs proportional to usage. The infrastructure was written in Terraform, organized into three layers (front-end, back-end and data), and the code is published by GitHub Actions pipelines. The React front-end is served by Amazon CloudFront from a private Amazon S3 bucket; authentication uses Amazon Cognito with the SRP protocol; an HTTP API Gateway validates the JWT before invoking a single AWS Lambda function, which stores data in Amazon DynamoDB and issues presigned URLs for direct photo uploads to S3. The results show that the solution meets the functional requirements, can be recreated in another account with a few commands and fully removed with a single command. Security and cost decisions, limitations and the mapping of services to Google Cloud are discussed, indicating the portability of the architecture in a multicloud context.

Keywords: cloud computing; serverless; infrastructure as code; Terraform; AWS.

1 Introdução

A computação em nuvem oferece recursos de computação sob demanda, por autoatendimento e com cobrança medida pelo uso (MELL; GRANCE, 2011). Dentro desse modelo, a computação serverless leva a abstração adiante: o desenvolvedor publica funções e consome serviços gerenciados, enquanto o provedor cuida da escala, da disponibilidade e da capacidade ociosa (BALDINI et al., 2017; JONAS et al., 2019).

O problema tratado neste trabalho é a construção de uma aplicação de quiz para encontros de estudo: cada encontro tem um questionário, os participantes enviam uma foto sobre o tema e disputam um ranking. Os requisitos não funcionais eram custo próximo de zero em períodos sem uso, nenhum servidor para administrar e a possibilidade de outra pessoa recriar o ambiente a partir do repositório.

O objetivo geral é apresentar a arquitetura, as decisões técnicas e a implementação do QuizOps na AWS. Os objetivos específicos são: (a) descrever as camadas da solução e o papel de cada serviço; (b) mostrar como a infraestrutura como código e a entrega contínua tornam o ambiente reproduzível; e (c) discutir segurança, custo, limitações e portabilidade para outro provedor.

O artigo está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico; a Seção 3, a metodologia; a Seção 4, a arquitetura e a implementação; a Seção 5, os resultados e a discussão; e a Seção 6, as considerações finais.

2 Referencial teórico

2.1 Computação serverless

BALDINI et al. (2017) definem serverless como um modelo em que o código roda em resposta a eventos, sem que o desenvolvedor provisione ou gerencie servidores, e em que a cobrança considera apenas o tempo de execução. JONAS et al. (2019) acrescentam que o modelo combina funções como serviço com serviços de back-end gerenciados, como bancos de dados e armazenamento de objetos, e apontam como vantagens a escala automática e o pagamento por uso.

Na AWS, o serviço de funções é o AWS Lambda, que executa o código em resposta a eventos e escala conforme a demanda (AMAZON WEB SERVICES, 2026j). O guia de boas práticas da AWS para aplicações serverless recomenda permissões mínimas por função, serviços gerenciados para estado e observabilidade desde o início (AMAZON WEB SERVICES, 2026i).

2.2 Infraestrutura como código

Infraestrutura como código (IaC) é a prática de definir a infraestrutura em arquivos versionados, aplicando a ela as mesmas técnicas usadas no desenvolvimento de software, como revisão, testes e controle de versão (FOWLER, 2016). O Terraform é uma ferramenta de IaC declarativa: o usuário descreve o estado desejado e a ferramenta calcula e executa as mudanças necessárias, por meio de provedores para diferentes plataformas (HASHICORP, 2026b). A existência de provedores para vários fornecedores de nuvem, como AWS e Google Cloud (HASHICORP, 2026a), torna a ferramenta adequada a cenários multicloud.

2.3 Integração e entrega contínuas

A entrega contínua automatiza o caminho entre uma mudança no repositório e sua publicação. O GitHub Actions executa fluxos de trabalho definidos em arquivos YAML, disparados por eventos do repositório, como um *push* (GITHUB, 2026b).

2.4 Autenticação baseada em tokens

O JSON Web Token (JWT) é um formato compacto para transmitir declarações assinadas entre duas partes (JONES; BRADLEY; SAKIMURA, 2015). O protocolo Secure Remote Password (SRP) permite que um cliente prove o conhecimento de uma senha sem enviá-la ao servidor, que armazena apenas um verificador derivado dela (WU, 1998). O Amazon Cognito implementa o SRP no fluxo de login e emite tokens JWT ao final da autenticação (AMAZON WEB SERVICES, 2026a).

3 Metodologia

Trata-se de uma pesquisa aplicada, conduzida como estudo de caso de desenvolvimento. O trabalho seguiu quatro etapas:

a) levantamento de requisitos em um documento de produto, com o fluxo de cadastro, quiz, mural e ranking;

b) registro das decisões técnicas em documentos curtos por tema (autenticação, mural, Lambda, API, dados, front-end, IaC e CI/CD), cada um com referência à documentação oficial;

c) implementação incremental por camada, primeiro em ambiente local (DynamoDB local e emulador de S3 em contêineres) e depois na AWS, com Terraform;

d) validação ponta a ponta na nuvem: cadastro, resposta ao quiz, envio de foto e consulta ao ranking.

As ferramentas utilizadas foram React 19 com Vite e Tailwind CSS no front-end, Node.js 24 no back-end, Terraform com o provedor AWS para a infraestrutura e GitHub Actions para a entrega. O código-fonte, a documentação e o passo a passo de reprodução estão disponíveis no repositório do projeto (BRITO, 2026b), e a demonstração em vídeo está nas referências (BRITO, 2026a).

4 Arquitetura e implementação

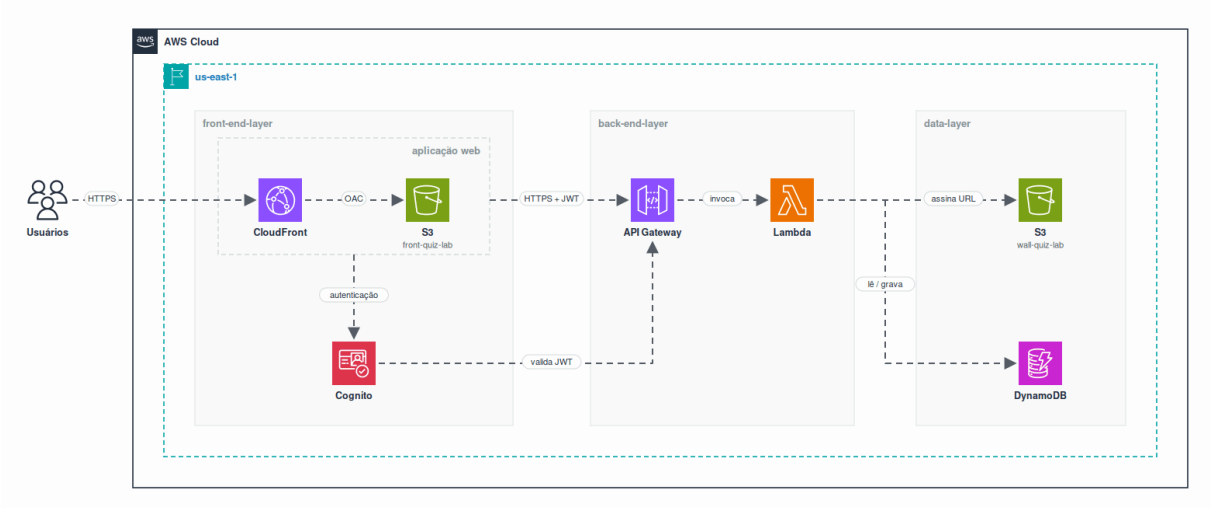
A Figura 1 mostra a arquitetura, dividida em três camadas dentro da região *us-east-1*. Todos os recursos seguem o padrão de nome `recurso-projeto-ambiente`, por exemplo `lambda-quiz-lab`, e recebem etiquetas padrão definidas no provedor do Terraform.

4.1 Camada de front-end

A aplicação React é compilada em arquivos estáticos e publicada em um bucket S3 privado. O Amazon CloudFront distribui esses arquivos por HTTPS no domínio padrão `*.cloudfront.net`. O acesso ao bucket usa o *Origin Access Control* (OAC): o CloudFront assina as requisições ao S3 e a política do bucket libera a leitura apenas para essa distribuição (AMAZON WEB SERVICES, 2026g). Como a aplicação é de página única, o erro 403 do S3 para rotas inexistentes é convertido em `index.html` com código 200.

O cadastro e o login usam um *user pool* do Amazon Cognito com um *app client* público, sem segredo, já que o código roda no navegador. O login segue o fluxo SRP, de modo que a senha não trafega na autenticação (WU, 1998; AMAZON WEB SERVICES, 2026a).

Figura 1 – Arquitetura do QuizOps na AWS



Fonte: elaborado pelo autor (2026).

4.2 Camada de back-end

Um API Gateway do tipo HTTP API expõe seis rotas. Um autorizador JWT, configurado com o emissor e a audiência do Cognito, valida o token de cada requisição: token ausente ou inválido recebe a resposta 401 sem que a função seja executada (AMAZON WEB SERVICES, 2026b). Todas as rotas usam uma única integração com uma função Lambda em Node.js 24, que identifica a rota pela combinação de método e caminho e lê o usuário das declarações do token.

A função recebe uma *role* do IAM com permissão para gravar logs e acessar apenas a tabela e o bucket do mural, seguindo o princípio do menor privilégio (AMAZON WEB SERVICES, 2026h). O grupo de logs no CloudWatch é criado pelo Terraform antes da função, com retenção de sete dias, para que também seja removido quando o ambiente é destruído.

4.3 Camada de dados

As respostas ficam em uma tabela do Amazon DynamoDB em modo sob demanda, cobrada por requisição (AMAZON WEB SERVICES, 2026f). A chave primária combina `quizId` (partição) e `userId` (ordenação) (AMAZON WEB SERVICES, 2026c), o que garante uma resposta por usuário em cada quiz; uma gravação condicional impede uma segunda tentativa. Os questionários ficam em arquivos JSON versionados junto ao código.

As fotos do mural ficam em um bucket S3 privado. O navegador não envia o arquivo à função: ele pede à Lambda uma URL pré-assinada, válida por cinco minutos, e envia a foto diretamente ao S3 (AMAZON WEB SERVICES, 2026d). A mesma técnica, com URLs de leitura, é usada para exibir as fotos. Assim, o tamanho do arquivo não esbarra no limite de carga

das invocações síncronas da Lambda (AMAZON WEB SERVICES, 2026e) e a função só é cobrada pelo tempo de assinar a URL.

4.4 Infraestrutura como código e entrega contínua

A infraestrutura fica em uma única pasta, com um arquivo por serviço, numerados na ordem de dependência, sem módulos. Duas variáveis locais, projeto e ambiente, compõem o nome de todos os recursos; alterá-las permite recriar o ambiente em outra conta sem conflito de nomes de bucket, que são únicos globalmente.

Dois fluxos do GitHub Actions publicam o código a cada *push* na branch principal, cada um disparado apenas por mudanças na sua pasta. O fluxo do front-end compila a aplicação, sincroniza os arquivos com o bucket e invalida o cache do CloudFront; o do back-end empacota o código com as dependências e atualiza a função. A infraestrutura permanece sob responsabilidade do Terraform, fora dos fluxos. O Quadro 1 resume as principais decisões.

Quadro 1 – Principais decisões técnicas

Decisão	**Justificativa**
Uma única função Lambda	Menos recursos para gerenciar; rotas resolvidas no código, todas com a mesma permissão.
HTTP API com autorizador JWT	Requisições não autenticadas são rejeitadas antes de gerar custo de execução.
URL pré-assinada para fotos	O arquivo não passa pela função, evitando o limite de carga e o custo de transferência.
DynamoDB sob demanda	Sem capacidade provisionada; custo zero sem requisições.
Buckets privados e OAC	Nenhum objeto é público; o acesso passa pelo CloudFront ou por URL assinada.
Domínio padrão do CloudFront	Dispensa domínio e certificado próprios, o que simplifica a reprodução.
Terraform sem módulos	Um arquivo por serviço, leitura linear e destruição completa com um comando.

Fonte: elaborado pelo autor (2026).

5 Resultados e discussão

A aplicação foi publicada e validada na AWS com o fluxo completo: cadastro com confirmação por e-mail, login, resposta ao quiz, envio e visualização de fotos e ranking por encontro e geral. Os dois fluxos de entrega executaram com sucesso, e uma alteração no front-end chega ao CloudFront sem passos manuais.

Quanto à reprodutibilidade, o repositório traz um passo a passo em que outra pessoa faz o *fork*, ajusta o perfil da AWS e o nome do projeto, aplica o Terraform e publica o código

pelos fluxos do GitHub Actions, sem instalar dependências da aplicação na própria máquina. O mesmo Terraform remove todos os recursos, inclusive buckets com arquivos e o grupo de logs.

Em segurança, a solução não expõe objetos públicos, não transmite a senha no login, valida o token na borda da API e restringe a função aos recursos que ela usa. Em custo, todos os serviços adotados cobram por uso, o que atende ao requisito de custo próximo de zero sem tráfego.

Foram identificadas as seguintes limitações:

a) o ranking geral usa uma leitura completa da tabela (*scan*), adequada ao volume de uma turma, mas que deve ser revista com muitos usuários;

b) os fluxos de entrega usam chaves de acesso estáticas guardadas como segredos do repositório; credenciais temporárias via OpenID Connect são a alternativa recomendada (GITHUB, 2026a);

c) o ambiente usa uma única região, sem estratégia de recuperação de desastres.

Do ponto de vista multicloud, cada serviço tem um equivalente gerenciado no Google Cloud, como mostra o Quadro 2. Como o Terraform também possui provedor para o Google Cloud (HASHICORP, 2026a), a mesma abordagem de IaC pode ser aplicada; a migração, porém, exige reescrever os recursos e adaptar o código que usa os SDKs de cada provedor.

Quadro 2 – Correspondência entre serviços da AWS e do Google Cloud

AWS	**Google Cloud**
Lambda	Cloud Run functions (GOOGLE CLOUD, 2026f)
API Gateway	API Gateway (GOOGLE CLOUD, 2026a)
Cognito	Identity Platform (GOOGLE CLOUD, 2026d)
DynamoDB	Firestore (GOOGLE CLOUD, 2026c)
S3 com URL pré-assinada	Cloud Storage com URL assinada (GOOGLE CLOUD, 2026e)
CloudFront	Cloud CDN (GOOGLE CLOUD, 2026b)

Fonte: elaborado pelo autor (2026).

6 Considerações finais

O trabalho apresentou o QuizOps, uma aplicação serverless completa na AWS, descrita em Terraform e publicada por entrega contínua. Os objetivos foram atingidos: as camadas e os serviços foram descritos, o ambiente pode ser recriado e removido a partir do repositório, e as decisões de segurança, custo e portabilidade foram discutidas.

Como trabalhos futuros, sugerem-se a troca das chaves estáticas por OpenID Connect nos fluxos de entrega, um índice ou tabela agregada para o ranking geral e a implementação

da mesma arquitetura no Google Cloud com Terraform, para comparar custo e esforço entre os provedores.

Referências

AMAZON WEB SERVICES. *Authentication flows*. 2026. Amazon Cognito Developer Guide. Disponível em: <https://docs.aws.amazon.com/cognito/latest/developerguide/amazon-cognito-user-pools-authentication-flow-methods.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Controlling access to HTTP APIs with JWT authorizers*. 2026. Amazon API Gateway Developer Guide. Disponível em: <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Core components of Amazon DynamoDB*. 2026. Amazon DynamoDB Developer Guide. Disponível em: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Download and upload objects with presigned URLs*. 2026. Amazon S3 User Guide. Disponível em: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Lambda quotas*. 2026. AWS Lambda Developer Guide. Disponível em: <https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *On-demand capacity mode*. 2026. Amazon DynamoDB Developer Guide. Disponível em: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/on-demand-capacity-mode.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Restrict access to an Amazon S3 origin*. 2026. Amazon CloudFront Developer Guide. Disponível em: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Security best practices in IAM*. 2026. AWS Identity and Access Management User Guide. Disponível em: <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *Serverless Applications Lens: AWS Well-Architected Framework*. 2026. Disponível em: <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/welcome.html>. Acesso em: 11 nov. 2026.

AMAZON WEB SERVICES. *What is AWS Lambda?* 2026. AWS Lambda Developer Guide. Disponível em: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>. Acesso em: 11 nov. 2026.

BALDINI, I. et al. *Serverless computing: current trends and open problems*. 2017. ArXiv:1706.03178. Disponível em: <https://arxiv.org/abs/1706.03178>. Acesso em: 11 nov. 2026.

BRITO, L. G. G. *QuizOps: demonstração da arquitetura serverless na AWS*. 2026. YouTube. 1 vídeo. Disponível em: <https://youtu.be/UdjNvdgt-6w>. Acesso em: 11 nov. 2026.

BRITO, L. G. G. *QuizOps: quiz serverless na AWS*. 2026. GitHub. Disponível em: <https://github.com/luizbrito7/gs02-multicloud.fiap>. Acesso em: 11 nov. 2026.

FOWLER, M. *Infrastructure as code*. 2016. Martinfowler.com. Disponível em: <https://martinfowler.com/bliki/InfrastructureAsCode.html>. Acesso em: 11 nov. 2026.

GITHUB. *OpenID Connect*. 2026. Disponível em: <https://docs.github.com/en/actions/concepts/security/openid-connect>. Acesso em: 11 nov. 2026.

GITHUB. *Understanding GitHub Actions*. 2026. Disponível em: <https://docs.github.com/en/actions/get-started/understand-github-actions>. Acesso em: 11 nov. 2026.

GOOGLE CLOUD. *About API Gateway*. 2026. Disponível em: <https://cloud.google.com/api-gateway/docs/about-api-gateway>. Acesso em: 11 nov. 2026.

GOOGLE CLOUD. *Cloud CDN overview*. 2026. Disponível em: <https://cloud.google.com/cdn/docs/overview>. Acesso em: 11 nov. 2026.

GOOGLE CLOUD. *Firestore overview*. 2026. Disponível em: <https://cloud.google.com/firestore/docs/overview>. Acesso em: 11 nov. 2026.

GOOGLE CLOUD. *Identity Platform documentation*. 2026. Disponível em: <https://cloud.google.com/identity-platform/docs>. Acesso em: 11 nov. 2026.

GOOGLE CLOUD. *Signed URLs*. 2026. Cloud Storage documentation. Disponível em: <https://cloud.google.com/storage/docs/access-control/signed-urls>. Acesso em: 11 nov. 2026.

GOOGLE CLOUD. *What is Cloud Run*. 2026. Disponível em: <https://cloud.google.com/run/docs/overview/what-is-cloud-run>. Acesso em: 11 nov. 2026.

HASHICORP. *Google Cloud Platform provider*. 2026. Terraform Registry. Disponível em: <https://registry.terraform.io/providers/hashicorp/google/latest/docs>. Acesso em: 11 nov. 2026.

HASHICORP. *What is Terraform?* 2026. Disponível em: <https://developer.hashicorp.com/terraform/intro>. Acesso em: 11 nov. 2026.

JONAS, E. et al. *Cloud programming simplified: a Berkeley view on serverless computing*. 2019. ArXiv:1902.03383. Disponível em: <https://arxiv.org/abs/1902.03383>. Acesso em: 11 nov. 2026.

JONES, M. B.; BRADLEY, J.; SAKIMURA, N. *RFC 7519: JSON Web Token (JWT)*. 2015. Internet Engineering Task Force. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519>. Acesso em: 11 nov. 2026.

MELL, P.; GRANCE, T. *The NIST definition of cloud computing*. Gaithersburg, 2011. Disponível em: <https://csrc.nist.gov/pubs/sp/800/145/final>. Acesso em: 11 nov. 2026.

WU, T. The secure remote password protocol. In: *Proceedings of the Network and Distributed System Security Symposium*. San Diego: Internet Society, 1998. Disponível em: <https://www.ndss-symposium.org/ndss1998/secure-remote-password-protocol/>. Acesso em: 11 nov. 2026.